

Containerize and Deploy the Python Service to Cloud Run

From a folder of Python to a sized, observable service — the proper way

WHAT WE'LL COVER

Five chapters, four labs

01 What a container actually is

02 Building a good image

03 The contract and the registry

04 Cloud Run's execution model

05 Operating revisions

01

What a container actually is

An image is stacked read-only layers plus metadata; a container is one process in that filesystem. See it before you build it.

01 What a container actually is

02 Building a good image

03 The contract and the registry

04 Cloud Run's execution model

05 Operating revisions

WHAT A CONTAINER ACTUALLY IS

An image is a stack of read-only layers plus metadata

config (JSON)

CMD, ENV, WORKDIR, USER, EXPOSE — no bytes, just metadata

layer 4 COPY app.py

your code — changes on every commit

layer 3 RUN pip install

site-packages — changes when requirements change

layer 2 COPY requirements.txt

one small file

layer 1 FROM python:3.13-slim

Debian + Python — shared with every image built on it

Three facts that explain everything else

- Layers are read-only and content-addressed — identical layers are stored and pulled once
- The cache invalidates top-down: change layer 2 and layers 3–4 rebuild
- The config, not a layer, says what runs: CMD, USER, PORT

THE RULE

An image is layers you can list plus a config you can read — if you cannot explain a layer, you did not need it.

WHAT A CONTAINER ACTUALLY IS

A container is one process in that filesystem — not a small VM

WORKED EXAMPLE — ONE PROCESS, AN EPHEMERAL WRITABLE LAYER

```
$ docker run -d --name orders -p 8080:8080 orders-api:1.0
9flc2e...
$ docker top orders
UID    PID    PPID   C   STIME  TTY   TIME      CMD
1000   41217  41196  0   10:02  ?     00:00:00  gunicorn --config gunicorn.conf.py
app:app
1000   41240  41217  0   10:02  ?     00:00:00  gunicorn --config gunicorn.conf.py
app:app
$ docker exec orders sh -c 'echo hello > /tmp/x; ls /tmp'
x
$ docker rm -f orders && docker run --rm orders-api:1.0 ls /tmp
# empty — the writable layer died with the container
```

A VM has	A container has
its own kernel, init, daemons	the host kernel; one process (PID 1)
a persistent disk	read-only layers + a writable layer that dies with it
boots in minutes	starts in milliseconds
logs in files	logs on stdout/stderr

Signals go to PID 1. If PID 1 is a shell that spawned your server, SIGTERM stops the shell and your server is killed 10 s later mid-request — chapter 3.

THE RULE

Design for a process, not a machine: stdout for logs, nothing on disk you need tomorrow, your server as PID 1.

02

Building a good image

Read a Dockerfile line by line: base image, pinning, multi-stage, non-root — and what each choice does to size and risk.

01 What a container actually is

02 Building a good image

03 The contract and the registry

04 Cloud Run's execution model

05 Operating revisions

WRITING A DOCKERFILE FOR A PYTHON SERVICE

Read it top to bottom — every line is a decision about a layer

WORKED EXAMPLE — A GENERIC ORDERS-API DOCKERFILE, ANNOTATED

```
FROM python:3.13-slim                # 1 base: slim, pinned major.minor
ENV PYTHONUNBUFFERED=1 PORT=8080    # config only — no layer
WORKDIR /app
COPY requirements.txt .              # 2 deps manifest first → cache
RUN pip install --no-cache-dir -r requirements.txt # 3 pinned deps
COPY app.py gunicorn.conf.py ./      # 4 code last: changes most
RUN useradd --create-home --uid 1000 appuser
USER appuser                         # 5 never root
EXPOSE 8080                          # documentation, opens nothing
CMD ["gunicorn", "--config", "gunicorn.conf.py", "app:app"] # 6 exec form: PID 1
```

SHELL FORM

- CMD gunicorn app:app → /bin/sh -c ... is PID 1; SIGTERM stops the shell and gunicorn is killed 10 s later

EXEC FORM

- CMD ["gunicorn", ...] → gunicorn itself is PID 1 and handles SIGTERM cleanly

THE RULE

Manifest first, code last, exec-form CMD, non-root — four habits that make every Python Dockerfile smaller, faster and safer.

BASE IMAGE CHOICE

Size and vulnerability surface are the same decision

Base	Size (approx.)	What is inside	Choose when	Trade-off
python:3.13	~1.0 GB	full Debian: gcc, make, docs, man pages	you must compile a C extension with no wheel	largest CVE surface, slowest pull
python:3.13-slim	~125 MB	Debian minimal + Python	pure-Python services — the default	no compiler: add build-essential in a builder stage only
python:3.13-alpine	~50 MB	Alpine + musl libc	size matters and every dependency has a musl wheel	musl breaks some wheels; slower Python
distroless python3	~55 MB	Python, libc, nothing else — no shell	hardened production, once the service is stable	no exec/shell for debugging; Debian-tracked Python version

Pin the tag you rely on: python:3.13-slim moves with patch releases (good); python:3-slim can jump a minor version under you; :latest is not a version.

THE RULE

Start on slim, move to distroless when the service is boring, and reach for the full image only for a compile you cannot avoid.

DEPENDENCY PINNING

Reproducible builds: the same Dockerfile, the same image next month

NOT REPRODUCIBLE

- RUN pip install flask gunicorn
- FROM python:3-slim
- Result: the image depends on the day it was built

REPRODUCIBLE

- requirements.txt: Flask==3.1.3, gunicorn==26.2.0
- FROM python:3.13-slim (or @sha256:... for bit-for-bit)
- Result: same inputs, same layers, same digest

WORKED EXAMPLE — THE TWO LINES THAT MAKE THE PIP LAYER REPEATABLE

```
$ cat requirements.txt
Flask==3.1.3
gunicorn==26.2.0
$ pip install --no-cache-dir -r requirements.txt      # inside the build
Successfully installed Flask-3.1.3 gunicorn-26.2.0 ...
# production step up: pip-compile requirements.in --generate-hashes → requirements.txt with every transitive pin + sha256
```

THE RULE

Pin direct dependencies with ==, lock the tree with hashes for production, and never let a build depend on today's PyPI.

MULTI-STAGE BUILDS

Build in one stage, run in another — the runtime image keeps only what runs

WORKED EXAMPLE — TWO STAGES, ONE RUNTIME IMAGE

```
FROM python:3.13-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.13-slim
COPY --from=builder /install /usr/local      # only the installed packages cross
over
WORKDIR /app
COPY app.py gunicorn.conf.py ./
...
```

What never reaches the runtime image

- compilers and headers (add build-essential to the builder only)
- pip's cache and downloaded sdist
- test and lint tooling
- anything deleted later in the builder

DOES NOT SHRINK AN IMAGE

- RUN apt-get install build-essential && pip install ... && apt-get remove build-essential — the bytes stay in the earlier layer; only a separate stage leaves them out

THE RULE

If a tool is needed to build but not to run, it belongs in a builder stage — deleting it later leaves it in the image.

RUNNING AS A NON-ROOT USER

Root in the container is root on the escape route — two lines remove it

WORKED EXAMPLE — TWO LINES, THEN PROVE IT

```
RUN useradd --create-home --uid 1000 appuser
USER appuser
EXPOSE 8080          # ≥ 1024: no root needed to bind

$ docker run --rm orders-api:1.0 id
uid=1000(appuser) gid=1000(appuser) groups=1000(appuser)
$ docker run --rm orders-api:1.0 sh -c 'touch /etc/x'
touch: cannot touch '/etc/x': Permission denied      # good
```

Needs root?	Fix
port 80 / 443	bind 8080; the platform terminates TLS
writing under /var or /etc	write to /tmp or a dir chown'd to the user
apt-get at runtime	install at build time, in the builder stage
reading a secret file	mount it readable by uid 1000

Cloud Run honours USER; if the named user does not exist in the image it runs as root (uid 0) — the useradd line is not optional.

THE RULE

useradd, USER, port ≥ 1024 — and verify with `id -u` before you push, because root is the default you get by forgetting.

Which layers rebuild when only app.py changes?

The hardened Dockerfile: FROM slim → COPY requirements.txt → RUN pip install → COPY app.py
gunicorn.conf.py → useradd → USER → CMD.

A) Only the COPY app.py layer

The cache is per instruction, independent of order.

B) COPY app.py and everything after it

The cache invalidates top-down from the first changed instruction.

C) Everything

Any change rebuilds the image from FROM.

D) Nothing — pip layer is the only real layer

Code copies are metadata.

LAB 1

Build the image twice — naive and hardened — and compare

LAB 1 · LAB MANUAL

Open the lab manual, Lab 1. Every command, console path and expected output is there — nothing is typed from this slide.

- Build the naive image; note size, layers, uid
- Build the hardened multi-stage image; compare the same four numbers
- Run it locally; call `/healthz` and `/whoami`
- Time a docker stop and read the shutdown line — the contract, measured

About 30 minutes
Kit: gcp-cloudrun-labs-all.zip
Cloud Shell only — no code

THE RULE

Two builds, four numbers, one clean stop — measure before and after, then move on.

03

The contract and the registry

What Cloud Run expects from your container — port, health, startup, shutdown — and how the image gets to Artifact Registry.

- 01 What a container actually is
- 02 Building a good image
- 03 The contract and the registry**
- 04 Cloud Run's execution model
- 05 Operating revisions

Port, health, startup, shutdown — what Cloud Run expects from PID 1

Clause	Cloud Run does	Your container must	In the kit
Port	injects PORT (default 8080), routes requests to it	listen on 0.0.0.0:\$PORT	gunicorn.conf.py: bind = 0.0.0.0:\$PORT
Startup	waits up to 4 min; holds requests until listening	start fast; answer the startup probe	slim image, 1 worker, /healthz
Health	probes startup / liveness (TCP by default, HTTP if configured)	expose a cheap endpoint with no dependencies	GET /healthz → {"status":"ok"}
Shutdown	SIGTERM to PID 1, then SIGKILL after 10 s	finish in-flight work within 10 s; exec-form CMD	graceful_timeout = 8; on_exit log line
Filesystem	in-memory; counts against the memory limit	write nothing you need; keep /tmp small	no writes
CPU between requests	throttled (request-based billing)	no background threads that expect CPU	none

THE RULE

Listen on \$PORT fast, answer /healthz cheaply, exit cleanly on SIGTERM inside 10 s, write nothing to disk — that is the whole contract.

Worked example: watching a clean shutdown — and a dirty one

EXEC-FORM CMD — CLEAN

- `$ time docker stop orders`
- `orders`
- `real 0m1.2s`
- `$ docker logs orders | tail -1`
- `orders-api: SIGTERM received, in-flight requests finished, exiting cleanly`
- `$ docker inspect orders --format '{{.State.ExitCode}}' → 0`

SHELL-FORM CMD — KILLED

- `$ time docker stop orders-shell`
- `orders-shell`
- `real 0m10.4s` ← waited the whole grace period
- `$ docker logs orders-shell | tail -1`
- `[INFO] Booting worker with pid: 8 (no shutdown line)`
- `$ docker inspect ... ExitCode → 137 (128 + SIGKILL 9)`

Read the clock: a stop that takes ~1 s handled SIGTERM; a stop that takes exactly the grace period was killed. On Cloud Run the grace period is 10 s and the killed request becomes a 5xx in your logs. Exit code 137 = 128 + 9 (SIGKILL); 143 = 128 + 15 (SIGTERM received but not handled).

THE RULE

Time the stop: about a second means clean; the full grace period means killed — and Cloud Run's grace period is 10 seconds.

BUILDING, TAGGING AND PUSHING

The image gets a registry address, then a home in Artifact Registry

THE ADDRESS

```
asia-south1-docker.pkg.dev / sandbox-rohan-07 / orders / orders-api : 1.0
└─ region + format host    └─ project      └─ repo    └─ image  └─ tag
```

ROUTE A — FROM CLOUD SHELL'S DOCKER

```
# A · local docker push
$ gcloud auth configure-docker asia-south1-docker.pkg.dev --quiet
$ docker tag orders-api:1.0 asia-south1-docker.pkg.dev/sandbox-rohan-07/orders/orders-api:1.0
$ docker push asia-south1-docker.pkg.dev/sandbox-rohan-07/orders/orders-api:1.0
1.0: digest: sha256:9e1d5c7a... size: 1786
```

ROUTE B — WHAT CI RUNS

```
# B · Cloud Build builds and pushes
$ gcloud builds submit --tag \
  asia-south1-docker.pkg.dev/sandbox-rohan-07/orders/orders-api:1.0
Step #0: Step 1/12 : FROM python:3.13-slim AS builder
... PUSH ... 1.0: digest: sha256:9e1d5c7a... size: 1786 STATUS: SUCCESS
```

After the push	Where
image + digest + tags	Artifact Registry → repo → image (console) · gcloud artifacts docker images list
vulnerability scan	Artifact Registry → image → Vulnerabilities tab (Artifact Analysis, enable the API once)

THE RULE

The tag is an address; the digest is the identity — push once, deploy by reference, and let the registry scan what you pushed.

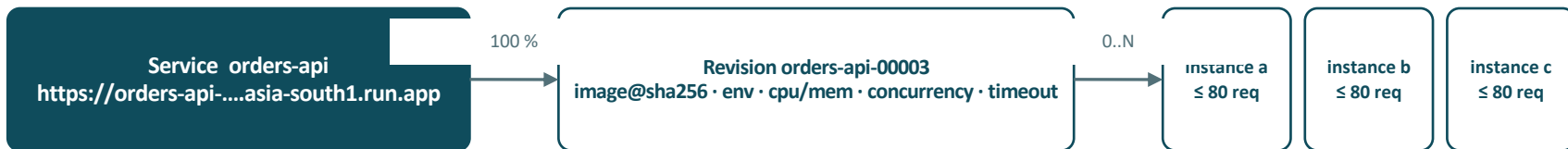
04

Cloud Run's execution model

Request-driven, stateless, scaled from zero: concurrency, CPU, memory, cold starts and timeouts, measured on your service.

- 01 What a container actually is
- 02 Building a good image
- 03 The contract and the registry
- 04 Cloud Run's execution model**
- 05 Operating revisions

Request-driven, stateless, scaled from zero — one picture



requests → instances: created on demand, each takes up to <concurrency> requests, removed when idle (to zero unless min-instances > 0)

Consequence	Because	You therefore
Stateless	any instance may answer; instances vanish	keep state in a DB/bucket; treat /tmp as scratch
Cold starts exist	a new instance must pull, start and pass the probe	keep the image small; min-instances for latency-sensitive paths
Concurrency is yours to set	one instance handles many requests at once	size CPU/memory for N concurrent requests, not one
CPU throttled between requests (default)	request-based billing	no background work — or switch to instance-based billing

THE RULE

A revision is what you deploy; an instance is what runs; requests are what you pay for — design the service around those three nouns.

Sizing an instance: how many requests share one container

Setting	Flag	Default	Range	How to choose
Concurrency	--concurrency	80	1–1000	match the server's real parallelism: threads/async for I/O-bound; ~1 per vCPU for CPU-bound
CPU	--cpu	1	<1 (with limits) ... 8	1 for most Python services; more only when a single request needs it
Memory	--memory	512Mi	up to 32Gi	peak RSS × concurrency headroom + anything written to /tmp
Billing mode	--cpu-throttling (default) · --no-cpu-throttling	request-based	—	instance-based when background work or min-instances warmth needs CPU
Max instances	--max-instances	100 (project quota applies)	—	cap the bill and protect downstreams

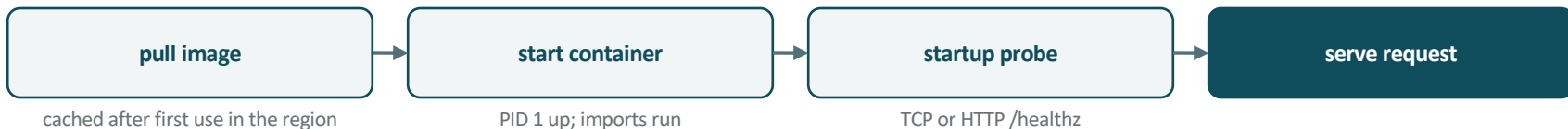
```
$ gcloud run deploy orders-api --image $AR/orders-api:1.0 --cpu 1 --memory 512Mi --concurrency 80 --max-instances 10
```

THE RULE

Concurrency is a promise about your server, not a wish — set it to what the process can really do in parallel, then size CPU and memory for that many.

COLD STARTS AND MINIMUM INSTANCES

A cold start is pull + start + probe — measure it before paying to remove it



Lever	Flag	Effect	Cost
Small image, lazy imports	(Dockerfile)	shorter start every time	none
Startup CPU boost	--cpu-boost	more CPU during start → faster imports	small, only during start
Minimum instances	--min-instances 1	no cold start while the warm instance lives	billed while idle
Instance-based billing	--no-cpu-throttling	warm instances keep CPU (background work)	higher per-instance rate

```
$ curl -s -o /dev/null -w 'total %{time_total}s\n' https://orders-api-...run.app/whoami # after 15 min idle
total 1.842s      # cold
total 0.061s     # warm, same command a second later
```

THE RULE

Measure the cold start before you pay to remove it — a smaller image is free, min-instances is a monthly bill.

REQUEST TIMEOUTS AND LONG-RUNNING WORK

The request timeout is a contract too — and where work goes when it will not fit

WORKED EXAMPLE — THE TIMEOUT IS ENFORCED BY THE PLATFORM

```
$ gcloud run services update orders-api --timeout 30
$ curl -s -o /dev/null -w '%{http_code} in %{time_total}s\n'
'https://orders-api-....run.app/slow?seconds=45'
504 in 30.02s
$ gcloud run services update orders-api --timeout 120
$ curl ... '/slow?seconds=45'
200 in 45.06s
```

Work takes	Put it	Why
< seconds	in the request; timeout $\approx$ p99 + margin	a tight timeout is a circuit breaker
minutes, caller waits	in the request; raise --timeout ($\leq$ 60 min)	only if a human is really waiting
minutes, caller does not wait	202 + Cloud Run job / Pub/Sub; status URL	the request returns, the work continues
hours / batch	Cloud Run job ($\leq$ 24 h per task, retries, parallel)	run-to-completion, no HTTP at all

THE TRAP

- Raising the timeout to 60 min for a background task: every client now waits, every instance is pinned for an hour, and a scale-in or a deploy kills it mid-way. A job survives all three.

THE RULE

Set the timeout to how long a caller should wait, not how long the work takes — work that outlives the caller belongs in a job.

YOUR CALL

Concurrency 80, one worker with 8 threads, CPU-bound image resizing — what happens under load?

Each request takes ~1 s of CPU. 80 arrive at once at one instance.

A) 80 finish in ~1 s

Concurrency means parallelism.

B) 8 run, 72 queue; latency climbs to ~10 s; few new instances

Cloud Run sees room for 80 and keeps routing to this instance.

C) Cloud Run starts 10 instances

It scales on CPU, not on concurrency.

D) Requests fail with 429

Overflow is rejected.

LAB 2

Push, deploy with sizing, provoke a timeout, measure a cold start

LAB 2 · LAB MANUAL

Open the lab manual, Lab 2. Every command, console path and expected output is there — nothing is typed from this slide.

- Cloud Build → Artifact Registry: the image gets its digest
- Deploy by image with cpu, memory, concurrency, max-instances, timeout
- Provoke the 504, then raise the timeout
- Measure a cold start; remove it with min-instances and CPU boost

About 30 minutes
Kit: gcp-cloudrun-labs-all.zip
Cloud Shell only — no code

THE RULE

Every sizing number is a flag you chose — the 504 and the cold start are the platform holding you to them.

05

Operating revisions

Revisions, traffic splits, environment configuration, logs and history — running the service after the first deploy.

01 What a container actually is

02 Building a good image

03 The contract and the registry

04 Cloud Run's execution model

05 Operating revisions

DEPLOYING A REVISION AND SPLITTING TRAFFIC

Every change is a revision; traffic is a routing decision you make separately

WORKED EXAMPLE — DEPLOY DARK, TEST BY TAG, CANARY, PROMOTE, ROLL BACK

```
$ gcloud run deploy orders-api --image $AR/orders-api:1.1 --no-traffic --tag canary
... revision [orders-api-00005-t3q] has been deployed and is serving 0 percent of traffic.
   canary: https://canary---orders-api-482915730261.asia-south1.run.app
$ curl -s https://canary---orders-api-482915730261.asia-south1.run.app/    # test in private
$ gcloud run services update-traffic orders-api --to-tags canary=10      # 10 % canary
$ gcloud run services update-traffic orders-api --to-latest              # promote
$ gcloud run services update-traffic orders-api --to-revisions orders-api-00004-k2d=100 # roll back
```

Flag	Effect
<code>--no-traffic</code>	new revision, 0 % — nothing changes for users
<code>--tag NAME</code>	a private URL for that revision
<code>--to-tags / --to-revisions</code>	weighted split; per-request, not sticky

Rollback is a traffic change to a revision that already exists — no build, no image pull, seconds.

THE RULE

Deploy dark, test by tag, move traffic by percentage, roll back by pointing — never by rebuilding.

RUNTIME CONFIGURATION AND ENVIRONMENT VARIABLES

Configuration lives outside the image — one image, many revisions

WORKED EXAMPLE — SAME IMAGE, NEW BEHAVIOUR, NEW REVISION

```
$ gcloud run services update orders-api --update-env-vars GREETING='Hello from v2',APP_VERSION=v2
✓ Deploying... Done.  revision [orders-api-00006-b8v] ... serving 100 percent of traffic.
$ curl -s https://orders-api-...run.app/
{"greeting":"Hello from v2","service":"orders-api","version":"v2"}
$ gcloud run services describe orders-api --format='yaml (spec.template.spec.containers[0].env) '
- name: GREETING
  value: Hello from v2
- name: APP_VERSION
  value: v2
```

Belongs in	Examples
env vars	feature flags, downstream URLs, log level, version label
Secret Manager (<code>--set-secrets</code>)	API keys, DB passwords, tokens
the image	code, pinned dependencies — nothing environment-specific
a config file (mounted volume)	large or structured config

NEVER

- `--set-env-vars DB_PASSWORD=hunter2` — it lands in shell history, revision metadata and the console. Use `--set-secrets DB_PASSWORD=db-password:latest`.

THE RULE

Build the image once, configure it per environment with env vars, keep secrets in Secret Manager — and remember each change is a revision.

READING LOGS AND REVISION HISTORY

stdout is your log file; the revision list is your change history

WORKED EXAMPLE — THE TWO LISTS YOU READ FIRST WHEN SOMETHING IS WRONG

```
$ gcloud run services logs read orders-api --region asia-south1 --limit 6
2026-09-07 10:41:02 GET 200 61ms /whoami
2026-09-07 10:41:02 GET 200 58ms /whoami
2026-09-07 10:39:48 slow request started: 45 s
2026-09-07 10:38:10 GET 504 30021ms /slow?seconds=45
2026-09-07 10:22:15 orders-api: SIGTERM received, in-flight requests finished,
exiting cleanly
$ gcloud run revisions list --service orders-api --region asia-south1
  REVISION      ACTIVE  SERVICE    DEPLOYED      DEPLOYED BY
✓ orders-api-00006-b8v  yes    orders-api  2026-09-07 10:40:11 UTC
you@example.com
✓ orders-api-00005-t3q      orders-api  2026-09-07 10:35:02 UTC
you@example.com
```

Where	Use for
gcloud run services logs read	quick tail from Cloud Shell; --log-filter for text
Console → service → Logs	filter by severity / revision; click into a request
Logs Explorer	cross-service queries, saved queries, alerts
Console → Revisions / gcloud run revisions list	what changed, when, by whom; describe to see config

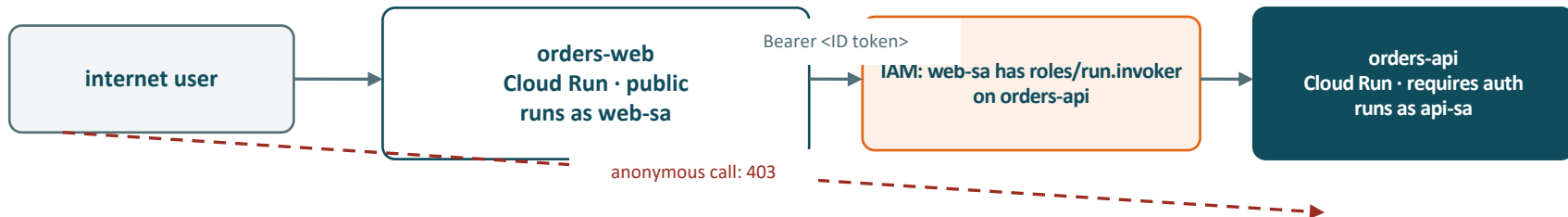
Request logs (status, latency, revision) are automatic; container logs are whatever you print — print JSON and Cloud Logging parses the fields.

THE RULE

Print to stdout, read by revision — the log line and the revision name together tell you what broke and which deploy did it.

SERVICE TO SERVICE

Two Cloud Run services talking: identity on the caller, a permission on the callee



WORKED EXAMPLE — THE CALLER MINTS A TOKEN, THE CALLEE'S IAM ALLOWS EXACTLY ONE CALLER

```
# caller side (already in the supplied web app — read, not written)
token = google.oauth2.id_token.fetch_id_token(auth_req, API_URL) # audience = the api's URL; from the metadata server
requests.get(f"{API_URL}/books", headers={"Authorization": f"Bearer {token}"})

# the one line of wiring
$ gcloud run services add-iam-policy-binding orders-api --member=serviceAccount:web-sa@... --role=roles/run.invoker
$ gcloud run services update orders-api --no-allow-unauthenticated --service-account api-sa@...
```

THE RULE

One identity per service, one invoker binding between them, no secret anywhere — that is how two Cloud Run services trust each other.

LAB 3

Configure by env var, split traffic, roll back, read the logs

LAB 3 · LAB MANUAL

Open the lab manual, Lab 3. Every command, console path and expected output is there — nothing is typed from this slide.

- A new revision from --update-env-vars alone
- 50/50 split, then a rollback by pointing traffic
- Logs and revision history, in gcloud and the console
- Teardown comes after Lab 4

About 20 minutes
Kit: gcp-cloudrun-labs-all.zip
Cloud Shell only — no code

THE RULE

Change the config, move the traffic, read the line — the operations you repeat every week.

LAB 4

Two services talking — web calls api with its own identity

LAB 4 · LAB MANUAL

Open the lab manual, Lab 4. Every command, console path and expected output is there — nothing is typed from this slide.

- Build the supplied bookshelf-web image with Cloud Build
- One service account per service; the api requires authentication
- Grant ONLY web's identity roles/run.invoker on the api
- Deploy web pointing at the api; open the page
- Remove the binding → the page fails; restore it → it works

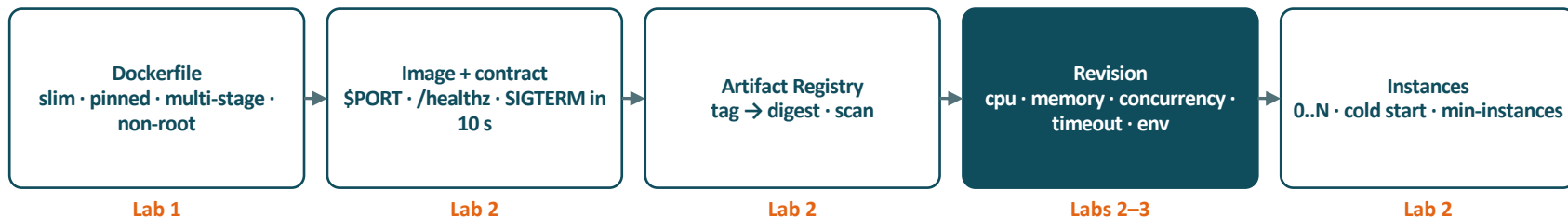
About 20 minutes
Kit: gcp-cloudrun-labs-all.zip
Cloud Shell only — no code

THE RULE

One identity each, one binding between them — and the page works or fails on that binding alone.

THE WHOLE PATH IN ONE PICTURE

Dockerfile → image → registry → revision → instance — where each stage landed



You can now	Because you did
explain an image from its layers	docker history on a naive and a hardened build
read a Dockerfile and predict its size, user and rebuild cost	built both and compared four numbers
state the container contract	timed a clean stop, read the shutdown line
size a Cloud Run service and defend the numbers	deployed with explicit flags, provoked a 504
measure and remove a cold start	cold vs warm, then min-instances + cpu-boost
operate a service without rebuilding	env-var revision, 50/50, rollback, logs, history

WHAT COMES NEXT

The service is containerized properly. The next sessions put things around it.

- The second service and private ingress — you already saw the shape in the landscape session; now both images come from today's Dockerfile.
- Secrets: the one thing that must not be an env var goes to Secret Manager with `--set-secrets`.
- The Claude-based service: the same revision shape — image, sizing, env, logs — plus a model path.
- CI: `gcloud builds submit` is the command a trigger runs on every push; the Dockerfile you read today is the one it builds.